

Архиватора на основе алгоритма Хаффмана

Ратушный Максим Витальевич

Группа: 2025.Б43-мм

Кафедра: *системного программирования*

Научный руководитель: *Литвинов Юрий Викторович*

Номер семестра практики: 2

Введение

Кодирование Хаффмана — классический алгоритм сжатия данных без потерь, применяемый во многих форматах (JPEG, PNG, DEFLATE и др.). В рамках учебной практики поставлена задача создать на языке Rust консольную утилиту, которая объединяет упаковку файлов и директорий со сжатием на основе кодирования Хаффмана.

Постановка задачи

Целью работы является разработка архиватора, выполняющего сжатие и распаковку файлов и каталогов с использованием алгоритма Хаффмана.

Для достижения цели необходимо решить следующие задачи:

1. Спроектировать бинарный формат архива, обеспечивающий упаковку иерархии файлов и каталогов в единый поток.
2. Реализовать алгоритм Хаффмана: построение дерева, таблицу кодовых путей, побитовый буфер и функции сжатия/распаковки.
3. Создать интерфейс командной строки (CLI) для упаковки и извлечения.
4. Выполнить функциональное тестирование с использованием property-based тестов.
5. Провести экспериментальное сравнение с распространёнными архиваторами (tar+gzip, zip) по времени выполнения и степени сжатия.

Обзор

Современные архиваторы, такие как ZIP, gzip и 7-Zip, обычно комбинируют словарное сжатие (LZ77/LZ78) со статистическим кодированием (Хаффмана или арифметическим). Это позволяет достигать высоких коэффициентов сжатия, но требует сложной реализации. Формат tar+gzip сначала формирует tar-архив (последовательность файлов с метаданными), а затем сжимает его с помощью gzip, использующего DEFLATE (LZ77 + Хаффман). В учебных целях выбран упрощённый подход: файлы объединяются в линейный поток посредством собственного формата (bundling), после чего весь поток сжимается чистым алгоритмом Хаффмана без предварительного LZ-сжатия. Это даёт возможность сосредоточиться на реализации алгоритма Хаффмана, работе с битовыми буферами и проектировании формата.

Язык Rust выбран из-за гарантий безопасности памяти, удобной системы сборки и богатой экосистемы тестирования, включающей библиотеку proptest для property-based тестирования, позволяющую автоматически проверять инварианты на большом количестве случайных данных. Использование Rust также повышает производительность.

Описание предлагаемого решения

Архитектура программы разделена на четыре модуля:

- `coding` — реализация кодирования Хаффмана: подсчёт частот байтов, построение дерева, таблица кодовых путей `PathTable`, побитовый буфер `BitBuffer` и функции `compress / decompress`.
- `bundling` — бинарный формат архива: последовательность записей, каждая из которых содержит длину относительного пути (`u16`), сам путь (байтовый вектор), флаг «директория» (`u8`, 0 — файл, 1 — директория), и для файлов — длину данных (`u64`) и данные. Конец архива маркируется записью с нулевой длиной пути.
- `lib` — интеграция двух предыдущих модулей: рекурсивный обход файловой системы, построение записей `ArchiveEntry` с вычислением относительных путей, а также функции для упаковки и распаковки.
- `main` — CLI, обрабатывающий ключи `-c` (создать архив) и `-x` (извлечь).

Алгоритм Хаффмана и побитовый ввод-вывод

Сжатие начинается с подсчёта частот всех 256 возможных байтов во входном потоке. Для полученных частот строится бинарное дерево с помощью очереди с приоритетом `BinaryHeap<Reverse<Box<HuffmanNode>>>`. Узлы сравниваются по частоте, а при равенстве — по наличию байта (внутренний узел считается меньше листа) и значению байта, что гарантирует детерминированное построение дерева. Если во входных данных присутствует только один уникальный байт, добавляется искусственный лист с нулевой частотой, чтобы дерево оставалось корректным.

Коды Хаффмана представлены не как целые числа фиксированной разрядности (что ограничило бы длину кода 64 битами), а в виде таблицы путей `PathTable`. Для каждого из 256 символов хранятся массив направлений (0 — левый потомок, 1 — правый) длиной до 255 элементов и длина пути. Таблица путей заполняется обходом дерева в глубину.

`BitBuffer` использует 128-битный аккумулятор и счётчик накопленных битов. Метод `write_bit` поочерёдно добавляет биты; при заполнении 8 бит младший байт сбрасывается в выходной вектор, что позволяет обрабатывать коды произвольной длины без переполнения. При декодировании заголовков архива (2048 байт частот и 8 байт исходной длины) используется для восстановления дерева, после чего входные биты последовательно проходят по дереву до достижения листа, где извлекается исходный байт. Процесс продолжается до получения заданного количества байт, после чего останавливается, игнорируя выравнивающие биты.

Формат файла

Сначала последовательность записей, каждая из которых начинается с 2-байтовой длины пути, затем байты пути, 1-байтовый флаг is_dir. Если запись является файлом, далее идут 8-байтовая длина данных и сами данные. Конец потока обозначается записью с длиной пути, равной нулю. Такой подход позволяет хранить произвольное количество файлов и папок с сохранением относительных путей. Благодаря этому при распаковке полностью восстанавливается исходная структура каталогов.

Тестирование и эксперименты

Функциональное тестирование

Для проверки корректности сжатия и формата использована библиотека proptest (для property-based тестирования). Генерируются сотни случайных последовательностей байт длиной до 10 Кбайт; для каждой проверяется тождество decompress(compress(data)) == data. Тесты успешно проходят. Аналогичные тесты написаны для bundling: случайный вектор записей ArchiveEntry сериализуется и десериализуется, после чего сравниваются поля всех записей. Оба набора тестов не выявили ошибок.

Экспериментальное сравнение с другими архиваторами

Для оценки эффективности и скорости работы проведены замеры на двух файлах: синтетическом test.txt и реальном файле script.py (код бенчмарка на Python). Сравнивались три инструмента: разработанный архиватор (huffman_archiver), tar+gzip (tar.gz) и zip. Для каждого инструмента выполнено 6 прогонов; первый считался прогревочным и отбрасывался, остальные 5 усреднялись. Измерялись время сжатия, время распаковки, размер полученного архива и коэффициент сжатия K, равный отношению размеров архива и исходных данных. Все испытания проводились на одной машине (AMD Ryzen 7, 16 ГБ ОЗУ, Linux 6.19.10-zen1-1-zen). Результаты каждого прогона и усреднённые показатели приведены в таблицах ниже.

Таблица 1: Все прогоны для файла test.txt

Инструмент	Прогон	Прогрев?	<i>T</i> , мс	<i>T</i> , мс	Размер, байт	<i>K</i>
huffman_archiver	1	да	6.573	3.172	20 656	0.4404
	2	нет	6.857	2.659	20 656	0.4404
	3	нет	9.986	1.574	20 656	0.4404
	4	нет	11.192	2.347	20 656	0.4404
	5	нет	9.411	2.276	20 656	0.4404
	6	нет	9.811	2.546	20 656	0.4404
tar.gz	1	да	6.172	5.704	500	0.0107
	2	нет	6.762	5.272	500	0.0107
	3	нет	6.048	5.174	500	0.0107
	4	нет	5.442	4.675	500	0.0107
	5	нет	5.304	4.819	500	0.0107
	6	нет	5.273	4.669	500	0.0107
zip	1	да	1.892	2.214	564	0.0120
	2	нет	1.884	2.078	564	0.0120
	3	нет	1.830	1.997	564	0.0120
	4	нет	1.892	1.969	564	0.0120
	5	нет	1.840	2.227	564	0.0120
	6	нет	1.843	2.181	564	0.0120

Таблица 2: Все прогоны для файла script.py

Инструмент	Прогон	Прогрев?	<i>T</i> , мс	<i>T</i> , мс	Размер, байт	<i>K</i>
huffman_archiver	1	да	1.907	1.302	5253	0.9174
	2	нет	1.883	1.325	5253	0.9174
	3	нет	1.825	1.320	5253	0.9174
	4	нет	1.719	1.259	5253	0.9174
	5	нет	1.900	1.225	5253	0.9174
	6	нет	2.143	1.289	5253	0.9174
tar.gz	1	да	5.214	4.725	1792	0.3130
	2	нет	5.269	4.705	1792	0.3130
	3	нет	5.282	4.956	1792	0.3130
	4	нет	5.191	4.757	1792	0.3130
	5	нет	5.364	5.405	1792	0.3130
	6	нет	5.328	4.765	1792	0.3130
zip	1	да	1.792	1.923	1843	0.3219
	2	нет	1.776	1.853	1843	0.3219
	3	нет	1.658	1.793	1843	0.3219
	4	нет	1.714	1.897	1843	0.3219
	5	нет	1.663	3.638	1843	0.3219
	6	нет	1.975	1.883	1843	0.3219

Анализ результатов

На синтетическом файле с повторяющимися последовательностями все три инструмента значительно уменьшают размер, однако huffman_archiver показывает коэффициент 0.44, в то время как tar.gz и zip достигают 0.01. Предположительно, причина в том, что чистое кодирование Хаффмана не способно компактно представить повторяющиеся цепочки символов. Методы LZ77 (используемые в DEFLATE) отлично сжимают дублирующиеся фрагменты, поэтому их результат на порядок лучше.

Таблица 3: Усреднённые показатели по 5 прогонам (прогревочный исключён)

Файл	Инструмент	Среднее T , мс	Среднее T , мс	Средний размер, байт	K
test.txt	huffman_archiver	9.45	2.28	20 656	0.440
	tar.gz	5.77	4.92	500	0.011
	zip	1.86	2.09	564	0.012
script.py	huffman_archiver	1.89	1.28	5253	0.917
	tar.gz	5.29	4.92	1792	0.313
	zip	1.76	2.21	1843	0.322

На скрипте `script.py` `huffman_archiver` даёт небольшое преимущество (0.92), а `tar.gz` и `zip` сжимают файл втрое (0.31–0.32). Это подтверждает, что для файлов с равномерным распределением байтов или малой избыточностью чистое статистическое кодирование малоэффективно. `huffman_archiver` хорошо показывает себя в плане скорости исполнения: на малых размерах `huffman_archiver` отрабатывает за единицы миллисекунд, практически не уступая `zip`, в то время как `tar.gz`, делающий два прохода, работает медленнее.

Разработанный архиватор демонстрирует корректность восстановления данных.

Заключение

В ходе выполнения работы получены следующие результаты:

- Реализован консольный архиватор на языке Rust, объединяющий упаковку файлов и каталогов со сжатием алгоритмом Хаффмана.
- Разработан собственный бинарный формат архива с поддержкой иерархии каталогов.
- Реализованы модули для побитового ввода-вывода, построения дерева Хаффмана и таблицы кодовых путей, гарантирующие корректную работу с кодами длиной до 255 бит.
- С помощью property-based тестов подтверждена корректность сжатия и формата на сотнях случайных входов.
- Проведено экспериментальное сравнение с `tar.gz` и `zip`, показавшее как ограничения чистого Хаффмана, так и приемлемую скорость работы на небольших данных.

Репозиторий: https://github.com/maxicot/huffman_archiver.

Документация: https://docs.rs/huffman_archiver/0.1.0/huffman_archiver.